

GNN Testing Notebook — Line-by-Line Walkthrough

Explains notebooks/gnn_testing.ipynb cell by cell: what each line does and what each output means, using live numbers from the weights on disk · generated 15 Sep 2026. Capstone: hybrid neuro-symbolic trading system, NSE Auto sector.

Background: say this first

What the GNN does. The graph has 15 auto-sector companies as nodes and 32 supplier→buyer links as edges. For a pair of companies A→B, the GNN predicts the probability that a shock at A today is followed by a shock at B within the next 5 trading days.

What a shock is. A one-day sector-excess return (stock minus Nifty Auto) larger than 2.5× the stock's own 63-day volatility, with a floor of 2%. It is measured against the sector index so that a day when the whole market moves does not count as a shock at every company at once, and against a 63-day volatility because a shorter window made the label predictable from volatility alone (a calm stock cleared its stale threshold as soon as normal volatility returned). Under the earlier definition a one-line "low volatility" rule beat every model out-of-sample; the redefinition is documented in config.yaml and the walk-forward report.

What "testing" means here. There are two kinds:

- **Correctness tests** in tests/gnn/ (77 of them). Each one has a right answer that is known in advance, so it either passes or fails.
- **Diagnostics.** Measurements of how good the model is. They have no pass/fail, so we look at them and interpret them.

The notebook doesn't define any tests itself. It calls the real test suite and the diagnostics script, then shows the results as tables and plots. That keeps one source of truth, which is a good design point to mention.

The metric, AUC. AUC (area under the ROC curve) measures ranking quality:

- It is the probability that a randomly chosen real cascade gets a higher score than a randomly chosen non-cascade.
- 1.0 means perfect ranking, 0.5 means random guessing, below 0.5 means worse than random — and a rule scoring 0.36 is a 0.64 rule read backwards, so simple rules are always quoted in their better direction.

Cell 1: Setup

```
import os, sys, subprocess, warnings
from pathlib import Path
warnings.filterwarnings("ignore")

# locate project root (folder containing config.yaml) whether run from notebooks/ or root
ROOT = next(p for p in [Path.cwd(), *Path.cwd().parents] if (p / "config.yaml").exists())
os.chdir(ROOT)                                # config.yaml paths (data/...) are root-relative
sys.path.insert(0, str(ROOT)); sys.path.insert(0, str(ROOT / "scripts"))

import pandas as pd
import matplotlib.pyplot as plt
import gnn_test_report as gtr                  # scripts/gnn_test_report.py
from src.common.config import load_config

cfg = load_config()
pd.set_option("display.width", 120)
print("project root:", ROOT)
```

- **Imports.** os handles folders, sys controls where Python looks for code, subprocess runs other programs, warnings controls warning messages; Path is a cleaner way to work with file paths. filterwarnings("ignore") hides library deprecation notices so the output stays readable — it changes no result.
- **ROOT = next(...)** finds the project root: it starts at the current folder, walks up through the parents, and stops at the first one containing config.yaml. So the notebook works whether it is opened from notebooks/ or from the root.

- **os.chdir(ROOT)** moves into the root because config.yaml uses relative paths like data/processed/... that only resolve from there. **sys.path.insert(...)** lets Python import from the project folder and from scripts/ — that is what makes *from src...* and *import gnn_test_report* work.
- **pandas** is tables, **matplotlib** is plots, **gnn_test_report** (short name gtr) is our own script that runs the tests and computes every diagnostic, and **load_config** reads config.yaml, which holds every setting (seed, model size, dates, the shock definition).
- The last three lines load the configuration into cfg, widen how tables print, and print the root folder as a sanity check.

Cell 3: Run the real test suite

```
summary, tests = gtr.run_pytest()
print(summary)
status = pd.Series([s for s, _ in tests]).value_counts()
display(status.to_frame("count"))
failed = [t for s, t in tests if s in ("FAILED", "ERROR")]
assert not failed, f"failing tests: {failed}"
pd.DataFrame(tests, columns=["status", "test"])
```

- **gtr.run_pytest()** runs *pytest tests/gnn* as a separate process (pytest is Python's standard testing tool). It returns the summary line and a list of (status, test name) pairs.
- The next lines print the summary, count how many tests have each status (PASSED / SKIPPED / FAILED) and show that as a small table.
- **assert not failed** means: stop the notebook with an error if any test failed. If the code is broken, the notebook refuses to continue and never shows plots based on broken code.
- The final expression shows every test and its status as a table.

Result

76 passed, 1 skipped in 7.35s: 76 passed, 1 skipped, 0 failed. Skipped: tests/gnn/test_train_utils.py:36: could not import 'sklearn.metrics': No module named 'sklearn'. The sklearn cross-check skips only when scikit-learn is not installed; nothing failed.

Cell 5: What data is being tested

```
d = gtr.diagnostics(cfg, seeds=[])
data = d["data"]
pd.Series({
    "timeline": f'{data["days"]} days, {data["date_range"]}',
    "nodes / features": f'{data["nodes"]} / {data["features"]}',
    "curated edges / msg-passing edges / scored pairs": f'{data["edges_curated"]} / {data["edges_mp"]} / {data["pairs_scored"]}',
    "train samples": f'{data["n_train"]} ({data["train_pos"]:.1%} pos) {data["train_range"]}',
    "val samples": f'{data["n_val"]} ({data["val_pos"]:.1%} pos) {data["val_range"]}',
}).to_frame("value")
```

- **gtr.diagnostics** does all the analysis in one call: loads the data and the trained model, scores the validation set, computes every metric shown below, and returns a dictionary d. **seeds=[]** means "don't retrain with other random seeds yet" — that is done in the last cell because it is slow. **data** is the part describing the dataset.
- The **pd.Series(...)** builds a two-column table; each row is explained below.

Result

Row	Value	Meaning
timeline	3635 days, 2011-10-03 -> 2026-06-30	trading days of data (history back to 2011)
nodes / features	15 / 7	15 companies; per company per day: logret_1, logret_5, logret_21, vol_21, mom_63, volz_21, present
edges	32 / 64 / 64	32 curated supplier→buyer links, doubled so information flows both ways, 64 directed pairs scored
train samples	4357 (16.9% positive)	examples the model learned from, 2012-01-06 -> 2023-04-10
val samples	1206 (16.8% positive)	held-out examples used to judge it, 2023-05-12 -> 2025-12-03

Point to make: validation is the latest period in time, with a 5-day gap before it, and no label may read a price after the configured label end date (2025-12-31, so the PPO test year never influences the GNN). The model is always tested on a future it never saw. A random split would let it "see the future", which is cheating. The *present* feature is 1 when a company is listed and 0 before (SONACOMS only lists in 2021).

Cell 6: Shock days per company

```
shocks = pd.Series(data["shock_days_per_ticker"])
shocks.plot.barh(figsize=(6, 4), title="Shock days per ticker (vol-scaled threshold)")
plt.gca().invert_yaxis(); plt.tight_layout(); plt.show()
```

- Takes the number of shock days per company and draws a horizontal bar chart. `invert_yaxis` puts the biggest bar on top, `tight_layout` stops labels overlapping, `show` displays the plot.

What it shows

Companies have between 34 and 110 shock days (most at 110, SONACOMS.NS lowest because it lists late). The counts are comparable because a shock is defined relative to each stock's own volatility — a naturally jumpy stock doesn't dominate the data.

Cell 8: The headline result, GNN vs simple rules

```
auc, base = d["auc"], d["baselines"]
scores = pd.Series({
    "GNN (val)": auc["val"],
    "GNN (train)": auc["train"],
    "baseline: train-period return corr": base["train_corr"],
    "baseline: dst prev-day vol (raw AUC)": base["dst_prev_vol"],
    "rule: dst prev-day vol, best direction": base["dst_vol_rule"], # a rule can be flipped for free
    "baseline: src prev-day vol (raw AUC)": base["src_prev_vol"],
})
ax = scores.plot.barh(figsize=(7, 3.8), title=f"AUC on the same {data['n_val']} val samples")
ax.axvline(0.5, color="k", ls="--", lw=1, label="chance"); ax.legend(); ax.invert_yaxis()
plt.tight_layout(); plt.show()
scores.round(4).to_frame("AUC")
```

- Pulls out the GNN's AUC scores and the baseline scores, puts them into one list so they can be plotted together, then draws a bar chart with a dashed line at 0.5 (random guessing).
- **What a baseline is:** a simple rule that involves no learning. If the GNN can't beat it, the GNN learned nothing useful. Everything is scored on the same 1206 validation samples, so the comparison is fair. The "rule" row is the volatility baseline in whichever direction scores higher, because a rule can always be flipped for free.

Result

	AUC	Meaning
GNN (val)	0.600	the real result
GNN (train)	0.672	score on data it trained on; gap +0.072 → no serious overfitting
Return correlation	0.527	"stocks that moved together in the past will cascade"
Buyer's volatility (raw)	0.487	"a jumpy buyer will shock", as scored
Buyer's volatility rule	0.513	the same rule in its better direction
Supplier's volatility (raw)	0.477	"a jumpy supplier causes shocks"

How to explain it: the GNN beats the volatility rule (+0.087) and beats the correlation rule (+0.073). The honest summary is "a real but modest signal". Mention why the rule is shown both ways: under the project's first label definition the buyer's volatility scored 0.36 here — which everyone read as "useless" until the out-of-sample check showed it was a 0.64 rule read backwards, better than any model. That is what triggered the label change.

Cell 9: Is it reliable over time?

```
monthly = pd.DataFrame(d["monthly"]).set_index("month")
fig, ax = plt.subplots(figsize=(8, 3.5))
xs = range(len(monthly))
ax.plot(xs, monthly["auc"], marker="o")
for i, (m, r) in enumerate(monthly.iterrows()):
    if r["auc"] == r["n"]:
        ax.annotate(f'n={r["n"]}', (i, r["auc"]), textcoords="offset points", xytext=(0, 6), ha="center",
                    fontsize=7)
ax.set_xticks(list(xs)); ax.set_xticklabels(monthly.index, rotation=45)
ax.axhline(0.5, color="k", ls="--", lw=1); ax.set_title("Val AUC by month (n on labels)")
plt.tight_layout(); plt.show()
monthly
```

- **monthly** is a table with one row per validation month: its AUC and sample count n. The plot draws a line of AUC per month with a dot for each month, and writes the sample count above each dot.
- **r["auc"] == r["n"]** is a trick for "this value is not NaN": NaN (a missing number) is the only value not equal to itself. A month whose samples are all one class has no AUC.
- The last lines label the x-axis with month names (tilted so they fit) and draw the 0.5 chance line.

What it shows

Monthly AUC swings from 0.20 (2025-10, n=21) to 0.87 (2025-03, n=47); 12 of 26 months are below 0.5 (2023-05, 2023-07, 2023-10, 2023-11, 2023-12, 2024-04, 2024-06, 2024-08, 2024-09, 2025-08, 2025-10, 2025-11). Monthly n runs 19–92; 2023-09, 2024-07, 2025-05, 2025-12 had a single class and no AUC. Some months have few samples, so part of this is noise. The conclusion is that the signal is intermittent, not steady. The trading agent has to learn how far to trust it.

Cell 11: Calibration — do higher scores mean more cascades?

```
cal = pd.DataFrame(d["calibration"])
fig, ax = plt.subplots(figsize=(5, 4))
ax.plot(cal["p_mean"], cal["actual"], marker="o", label="model")
ax.plot([0, 1], [0, 1], "k--", lw=1, label="perfect calibration")
ax.set_xlabel("mean predicted p (quintile)"); ax.set_ylabel("actual cascade rate")
ax.set_title("Calibration on val"); ax.legend(); plt.tight_layout(); plt.show()
cal
```

- The validation predictions were sorted into 5 equal groups (quintiles) by predicted probability. For each group we have the average predicted probability and the actual fraction that cascaded.
- The plot shows predicted against actual. The dashed diagonal is where a perfectly calibrated model would sit (predicts 30% → 30% actually happen).

What it shows

The lowest-scored group cascaded 10% of the time, the highest 25%; the ordering is monotone — correct. The curve sits away from the diagonal (mean predicted 0.40 against a 17% base rate) because training re-weights the rare positive examples, which shifts all predicted probabilities. That was deliberate. Use the score as a ranking, not as a literal probability.

Cell 13: Which inputs does the model use?

```
imp = pd.Series(d["feature_importance"])
imp.sort_values().plot.barh(figsize=(6, 3), title="Val AUC drop when feature is shuffled")
plt.axvline(0, color="k", lw=1); plt.tight_layout(); plt.show()
```

- Plots permutation importance for every model input: the node features and, when configured, the pair-head inputs (prefixed pair:).
- **How permutation importance works:** take one input, randomly shuffle its values across days (which destroys any information it carries), and re-measure validation AUC. The drop in AUC tells you how much the model relied on that input.

Result

Input	AUC drop
volz_21	+0.078
logret_21	+0.035
vol_21	+0.033
logret_5	+0.001
present	+0.000
pair:edge_weight	+0.000
pair:direction	+0.000
mom_63	-0.000
pair:rolling_corr	-0.002
logret_1	-0.004

How to explain it: the model leans most on **volz_21** (+0.078). Inputs near zero are not necessarily useless — shuffling one at a time understates an input that carries the same information as another (the pair's edge weight and direction, for instance, never change over time, so the model can absorb them into its weights). It is a guide to what the model relies on, not a verdict.

Cell 15: Per-pair view

```
pp = pd.DataFrame(d["per_pair"])
print(f"{len(pp)} evaluable pairs, {int((pp['auc'] < 0.5).sum())} below 0.5")
pd.concat({"top 5": pp.head(5), "bottom 5": pp.tail(5)})
```

- **pp** has the AUC for each supply-chain pair. It only includes pairs with at least 8 validation samples and both outcomes present, since otherwise AUC can't be computed.
- It prints how many pairs could be scored and how many fall below 0.5, then shows the best 5 and worst 5.

Result

61 pairs scored, 14 below 0.5; the weak ones most often involve **BOSCHLTD.NS** (5 of them). Each pair has only a handful of samples, so treat this as a hint worth investigating (are that company's edges in the graph correct?), not as proof.

Cell 17: The no-look-ahead test (the most important one)

This checks that the model can't see the future. If tomorrow's data leaked into today's score, every backtest result would be fake.

```
import copy, torch
from src.common.seeding import set_global_seed
from src.models.gnn.graph_dataset import build_dataset
from src.models.gnn.train import WEIGHTS_DIR, forward_all_pairs, make_model

set_global_seed(cfg.project.seed)
gd = build_dataset(cfg)
model = make_model(cfg, gd) # same shape as the shipped weights (node, pair, context
                             dims)
model.load_state_dict(torch.load(WEIGHTS_DIR / "propagation_gnn.pt", weights_only=True)); model.eval()
W, t = cfg.gnn.temporal_window_days, 300

with torch.no_grad():
    before = forward_all_pairs(model, gd, t, W)
    noisy = copy.copy(gd) # trash every input AFTER day t
    noisy.x = gd.x.clone(); noisy.x[t + 1:] = torch.randn_like(noisy.x[t + 1:]) * 10
    if gd.pair_feat is not None:
        noisy.pair_feat = gd.pair_feat.clone(); noisy.pair_feat[t + 1:] = torch.randn_like(noisy.pair_feat[t
            + 1:])
    after_future = forward_all_pairs(model, noisy, t, W)
    x = gd.x.clone(); x[t - 3] += 5.0 # nudge a day INSIDE the window
    after_past = forward_all_pairs(model, gd, t, W, x=x)

print("day", gd.dates[t].date())
print("max |Δ| after corrupting the future :", float((before - after_future).abs().max()), "-> must be 0")
print("max |Δ| after nudging day t-3      :", float((before - after_past).abs().max()), "-> must be > 0
      (window is used)")
```

- **Imports:** `copy` (to make a shallow copy of the dataset object), `PyTorch`, the seeding function, the dataset builder, and three helpers from the training module: the weights folder, **`make_model`** (builds a model with exactly the input sizes the dataset has — node features, pair-head inputs, market context) and **`forward_all_pairs`** (scores all directed pairs on one day, feeding the model the trailing window, the edges with their weights, and that day's pair inputs).
- **`set_global_seed`** fixes all randomness to seed 42; **`build_dataset`** builds the [day, company, feature] array plus the per-day pair inputs; the model is created, the trained weights are loaded from disk, and **`eval()`** switches off dropout so outputs are deterministic.
- **`W = 10`:** the model looks at the last 10 days. **`t = 300`:** the day we test on (2012-12-26). **`no_grad()`** means "just predict, don't prepare for training".
- **`before`** scores all pairs on day `t` from clean data. Then a copy of the dataset has every day AFTER `t` replaced with large random noise — node features and pair inputs alike — and day `t` is scored again as **`after_future`**. If the scores change at all, the model was using future data.
- **Control experiment:** start from clean data again and nudge one day inside the window (day `t-3`). This time the output should change. That proves the test above isn't passing only because the model ignores its input.
- The prints show the largest change across all pair scores for each experiment.

Result

- Future destroyed → change = 0.0 exactly. There is no leakage.
- Past day nudged → change = 0.049. The model does use its 10-day history.

Together these prove, rather than assume, that the scores are leak-free. The same check runs as a `pytest` test, and separate tests prove the inputs themselves (node features, market context, rolling pair correlation) are computed without look-ahead.

Cell 19: Seed robustness

```
SEEDS = [7, 2026]
if SEEDS:
    ds = gtr.diagnostics(cfg, seeds=SEEDS)
    meta = ds["meta"]
```

```

rows = [{"seed": meta["seed"], "val_auc": meta["best_val_auc"], "best_epoch": meta["best_epoch"],
        "note": "shipped"}]
rows += [{"**r", "note": "retrained (temp dir)"} for r in ds["seeds"]]
seeds_df = pd.DataFrame(rows).set_index("seed")
seeds_df["val_auc"].plot.bar(figsize=(4, 3), title="Val AUC by seed", ylim=(0.5, 0.7), rot=0)
plt.tight_layout(); plt.show()
display(seeds_df)
else:
    print("skipped")

```

- Retrains the GNN from scratch with other random seeds. It writes to a temporary folder, so the real trained weights are never touched. Takes about a minute per seed.
- Builds a table: the first row is the model we actually use (read from its saved training record), then one row per retrained seed; then a bar chart and the table. Setting `SEEDS = []` skips the cell.
- **Why this matters:** neural networks start from random weights, so a different seed gives a slightly different model. This checks whether the headline number was a lucky or unlucky draw.

Result

Seed	Val AUC	Best epoch
42 (used)	0.600	3
7	0.604	35
2026	0.605	31

How to explain it: results span 0.600–0.605 (spread 0.005); our seed ranks 3 of 3. We keep seed 42 because the project fixes the seed for reproducibility, and we report the AUC as a range rather than to three decimals.

Last cell: Takeaways (text)

- The code is correct: all tests pass.
- The signal is real but modest: 0.600 against 0.527 for the correlation rule and 0.513 for the volatility rule.
- The model relies most on `volz_21`; near-zero inputs may be redundant rather than useless.
- The signal varies month to month.
- There is provably no future leakage, and the cached scores reproduce exactly.
- Out-of-sample (walk-forward, five half-year windows, 183 shock days): the shipped configuration scores 0.584 (95% CI 0.535–0.636) against 0.525 for the volatility rule. This, not the single validation split, is the number to defend.

Likely teacher questions

"Why not just use accuracy?"

Only about 17% of examples are cascades. A model that always says "no cascade" would be 83% accurate and useless. AUC measures ranking and isn't fooled by that imbalance.

"Isn't 0.6 low?"

Predicting stock moves is hard. What matters is that it beats the no-learning rules on identical data, out of sample, and that the margin is quoted honestly.

"How do you know it's not cheating?"

Cell 17. We destroyed all future data and the output didn't change at all, and we showed with a control that the model does react to real inputs. Separate tests do the same for every input series.

"Why did the definition of a shock change?"

Because a one-line rule — rank pairs by the buyer's low recent volatility — beat every model under the first definition (0.64 AUC out of sample). A 21-day volatility mean-reverts inside the 5-day label window, so calm

stocks cleared their stale threshold as soon as normal volatility returned. Measuring shocks net of the sector index against a 63-day volatility removes that shortcut (the rule drops to about 0.52). The change was made before the final model was selected, and the old results are archived, not deleted.

"Why is the curve away from the diagonal in calibration?"

Class weighting during training shifts all probabilities on purpose. The ordering is what the trading agent uses.

"Why keep a seed that isn't the best?"

Reproducibility. The seed was fixed before looking at results; choosing the best seed afterwards would be cherry-picking.